

UNIT-3

Models Based on Decision Trees: Decision Trees for Classification, Impurity Measures, Properties, Regression Based on Decision Trees, Bias–Variance Trade-off, Random Forests for Classification and Regression. The Bayes Classifier: Introduction to the Bayes Classifier, Bayes’ Rule and Inference, The Bayes Classifier and its Optimality, Multi-Class Classification, Class Conditional Independence and Naive Bayes Classifier (NBC)

Models Based on Decision Trees:

Introduction to Decision Trees :Decision tree is a popular structure in ML. It is built by splitting the data set associated with a node into subsets that typically have less entropy or better purity. Splitting is carried out using the values of a selected feature; this feature-based decision-making at each of the internal nodes offers transparency and is ideally suited for explanation and for easier understanding by application domain experts.

Some properties of the decision tree (DT) building algorithm are:

It is greedy: The best feature, in best feature, in terms of purity in splitting, among the given set of features is invariably used at the root node of the DT. So, the feature used at each of the child nodes will depend on the feature selected at the parent node. Such a sequential greedy process may fail to be optimal on the whole.

It can be expensive: Selection of a feature depends both on the number of features and the number of patterns. So, tests conducted on the feature values at each node need to be simple. Even simple tests can become computationally demanding when the dimensionality is large.

It can overfit : The depth of the DT can increase as more splits are considered sequentially. Such a deep DT can overfit the training data and perform poorly on the validation/test data. A solution to this problem is provided by building the DT using the training data without worrying about depth and then pruning the more recent subtrees based on its performance on a validation data set. This process is called pruning and it helps in solving the overfitting problem.

Random Forest: It is a collection of decision trees. The model predicts by combining the predictions of the trees in the collection.

AdaBoost: It combines multiple weak learners to realize a good learning algorithm; a weak learner is a learner that provides more than 50% accuracy. One can use a variety of ML models as possible weak learners; however, simple DT-based weak learners are popular.

Gradient Boost: It is based on a sequence of ML models where each model is built to correct the error in the prediction of the previous model. Here also, it is possible to use any ML model in the sequence, but again, DT models are more popular. In this case, learnt models are used one after the other to predict the overall outcome.

Decision Trees for Classification: A decision tree is a tree with a root node and each child of the root node forms the root node of the subtree below it. We associate the entire training data set with the root node. The process of building the decision tree depends upon splitting the data set associated with a node.

A popular approach is to split, based on a feature, the set of patterns associated with a node so that the resulting subsets are least impure. These subsets are associated with the children nodes of the node. Impurity of the split at a node is captured using an impurity measure; the most popular one is **Shannon's entropy**. It is given by:

$$-\sum_{i=1}^C p_i \log(p_i),$$

where p_i is the probability of class i , $i = 1, 2, \dots, C$ assuming that there are C classes. These probabilities are typically estimated based on the fraction of elements from each class among the set of patterns associated with the node.

Five different probability distributions and their entropy values

Case	Class 1 (p_1)	Class 2 (p_2)	Entropy
1	100 ($p_1 = 1.0$)	0 ($p_2 = 0.0$)	0.0
2	75 ($p_1 = 0.75$)	25 ($p_2 = 0.25$)	0.811
3	50 ($p_1 = 0.5$)	50 ($p_2 = 0.5$)	1.0
4	25 ($p_1 = 0.25$)	75 ($p_2 = 0.75$)	0.811
5	0 ($p_1 = 0.0$)	100 ($p_2 = 1.0$)	0.0

Here, we use the fact that the limit of $p \log(p)$ is 0 as $p \rightarrow 0$ in cases 1 and 5 in the table where the entropy is 0. Some interesting observations from the table are:

Entropy is non-negative; it is greater than or equal to zero.

This is because $p_i \geq 0$ and $\log(p_i) \leq 0$ for all i .

So, $-p_i \log(p_i) \geq 0$ for all i .

As a consequence, if there are C classes, then

$$-\sum_{i=1}^C p_i \log(p_i) \geq 0$$

So, the minimum possible value is **0**.

In a two-class scenario, entropy is symmetric, that is:

$$\text{entropy}(p_1, p_2) = \text{entropy}(p_2, p_1)$$

This is because:

$$\text{entropy}(p_1, p_2) = -p_1 \log(p_1) - p_2 \log(p_2) = -p_2 \log(p_2) - p_1 \log(p_1) = \text{entropy}(p_2, p_1)$$

In a two-class problem, entropy is maximum when $p_1 = p_2 = 0.5$.

This can be explained by computing the partial derivatives of entropy and equating them to 0.

The partial derivative of $entropy(p_1, p_2) = -p_1 \log(p_1) - p_2 \log(p_2)$ with respect to p_1 is:

$$\frac{\partial entropy(p_1, p_2)}{\partial p_1} = -1 - \log(p_1)$$

Similarly, with respect to p_2 , it is:

$$-1 - \log(p_2)$$

Similarly, with respect to p_2 it is,

$$-1 - \log(p_2)$$

If we equate these two quantities to 0, then we get:

$$-1 - \log(p_1) = -1 - \log(p_2) = 0 \Rightarrow \log(p_1) = \log(p_2) \Rightarrow p_1 = p_2$$

However, $p_1 + p_2 = 1$ and if $p_1 = p_2$ then:

$$p_1 = p_2 = 0.5$$

This corresponds to the **maximum entropy**, as:

$$\frac{\partial^2 entropy(p_1, p_2)}{\partial p_1^2} = \frac{-1}{p_1} \quad \text{and} \quad \frac{\partial^2 entropy(p_1, p_2)}{\partial p_2^2} = \frac{-1}{p_2}$$

Both are negative since $p_1, p_2 \geq 0$, $p_1, p_2 \geq 0$.

Also

$$\frac{\partial^2 entropy}{\partial p_1 \partial p_2} = 0$$

Thus, the Hessian matrix H is:

$$H = \begin{bmatrix} -2 & 0 \\ 0 & -2 \end{bmatrix}$$

When $p_1 = p_2 = 0.5$, since:

$$-\frac{1}{p_1} = -\frac{1}{p_2} = -2$$

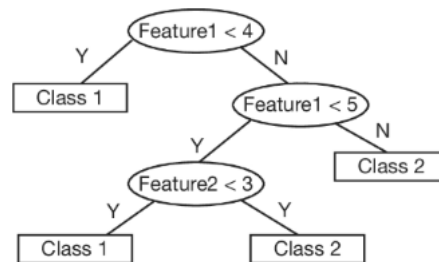
- A matrix $A_{C \times C}$ is **negative definite** if $x^T A x < 0$ for any real vector x of size $C \times 1$.
So, in this case, the Hessian matrix H is negative definite as:

$$x^T H x = -2x(1)^2 - 2x(2)^2$$

So, the value $p_1=p_2=0.5$ corresponds to the **maximum entropy**.

Entropy depends on the probability distribution — the values of p_1 and p_2 in a two-class case.

However, for the sake of brevity, we use entropy without arguments when the context is clear.



Decision tree for the data set

Impurity Measures, Properties:

Construction Even though shannon's entropy is popular for capturing the impurity associated with a split, there are other impurity measures. Here, by p_i , we mean the probability of the i th class. Some of the important ones are: Gini Index: The impurity at a node for a C class problem is defined as :

$$1 - \sum_{i=1}^C p_i^2$$

Misclassification Impurity: It is defined as

$$1 - \max_j p_j$$

Consider a node at which the associated set has 80% of patterns from Class 1, 15% from Class 2 and 5% from Class 3. So, $p_1 = 0.8$, $p_2 = 0.15$, $p_3 = 0.05$. Then, the misclassification impurity of the node is $1 - 0.8 = 0.2$.

Note that it is the sum of the two smaller probabilities p_2 and p_3 . Note that entropy, Gini index and misclassification impurity are all impurity measures. They can be used to find the impurity of a node. We can also use them in computing the impurity associated with a split by considering the weighted impurity used in Example 2. In the same example, we used entropy as the impurity measure.

example data set shown in Table . There are three independent features: Outlook, Temperature in degrees Fahrenheit, and percentage Humidity. For the classes, there are two possibilities: P standing for Play and NP standing for No Play.

A data set used to illustrate properties of decision trees

Pattern Outlook Temp(F) Humidity Class

1	Sunny	65	91	NP
2	Sunny	78	70	P
3	Overcast	85	82	P
4	Overcast	62	71	P
5	Rainy	70	65	NP
6	Rainy	79	62	P

If we use **Outlook** for splitting, based on its value, we will obtain three subsets: **{1,2}**, **{3,4}**, and **{5,6}**.

The second subset is *pure*:

$p_1=1$ and

$p_2=0$

The **Gini index** value for this subset is:

$$0 (1 - 1^2 - 0^2 = 0)$$

In the cases of the first and third subsets:

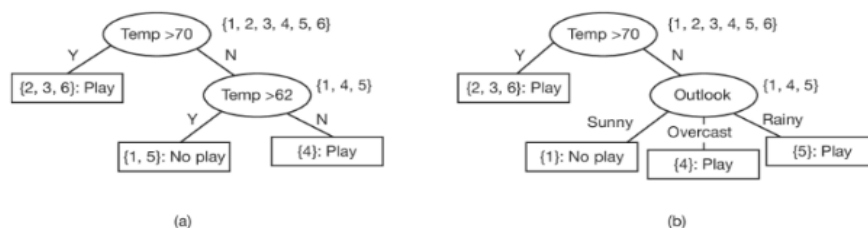
$$p_1=p_2=0.5$$

So, the Gini index value for each of them...is $1 - (0.5)^2 - (0.5)^2 = 0.5$ and each subset has 2 out of 6 elements. So, the weights are each.

So, the Gini impurity value for the whole split is $\frac{2}{6} \times 0.5 + \frac{2}{6} \times 0.5 = \frac{1}{3}$. So, using Outlook for splitting, we get as the impurity value. If we use Humidity for splitting and use the test Humidity < 91, then we get two subsets, {1} and {2,3,4,5,6}.

The first subset {1} is pure and has 0 Gini index value. For the second set, $p_1 = \frac{5}{6}$ and $p_2 = \frac{1}{6}$. So, the Gini index value is $1 - (\frac{5}{6})^2 - (\frac{1}{6})^2 = \frac{5}{6}$ and the weight is as 5 out of 6 patterns are present in the second subset. So, the weighted value is

If we perform the split based on Temperature > 70, at the root, we obtain a pure subset {2,3,6} and the second subset {1,4,5} needs to be split further. we have two options for splitting the set {1,4,5}. One is based on Temperature >62 and other is based on Outlook both of them lead to the same impurity value of 0



Decision trees for the data set

Regression Based on Decision Trees :

Regression and classification are related predictive tasks .

Example : Let us consider the data set we have used to illustrate KNN regression in the previous chapter; we show it in Table 3.5. There are **9 two-dimensional patterns** and the respective target values. We call the features f_1 and f_2 .

Number	f_1	f_2	Target (y)
1	0.2	0.4	8
2	0.4	0.2	8
3	0.6	0.4	12

Number	f_1	f_2	Target (y)
4	0.8	0.6	16
5	1.0	0.7	19
6	0.8	0.4	14
7	0.6	0.2	10
8	0.5	0.5	12
9	0.2	0.6	10

(SE). Let a set $\mathbf{S}$ of $\mathbf{n}$ elements be split into sets $\mathbf{S}_1$ and $\mathbf{S}_2$ having $\mathbf{n}_1$ and $\mathbf{n}_2$ elements, respectively. Further, let the sample means of the target ($\mathbf{y}$) values of the sets $\mathbf{S}_1$ and $\mathbf{S}_2$ be μ_1 and μ_2 , respectively. Then the **weighted squared error** for the split is:

$$\frac{n_1}{n} \sum_{x_i \in S_1, i=1}^{n_1} (y_i - \mu_1)^2 + \frac{n_2}{n} \sum_{x_i \in S_2, i=1}^{n_2} (y_i - \mu_2)^2$$

Note that:

$$\mu_1 = \frac{1}{n_1} \sum_{x_i \in S_1, i=1}^{n_1} y_i \quad \text{and} \quad \mu_2 = \frac{1}{n_2} \sum_{x_i \in S_2, i=1}^{n_2} y_i$$

For f_1 , we need to consider splits based on

$f_1 < 0.4$, $f_1 < 0.5$, $f_1 < 0.6$, $f_1 < 0.8$, $f_1 < 1.0$, $f_1 \leq 1.0$.

The respective SEs are given in **Table**

Test	SE of S_1	SE of S_2	Weighted SE
$f_1 < 0.4$	2	82	64.22

$f_1 < 0.5$	2.67	52.93	36.11
$f_1 < 0.6$	11	48.8	32.04
$f_1 < 0.8$	16	12.66	14.9
$f_1 < 1.0$	55.5	0	49.33
$f_1 \leq 1.0$	108	0	108

Note that the minimum value of weighted SE, that is, **14.9**, is obtained for the split based on the test $f_1 < 0.8$. Subsets S_1 and S_2 for this optimal split are

$$S_1 = \{1, 2, 3, 7, 8, 9\} \text{ and } S_2 = \{4, 5, 6\}$$

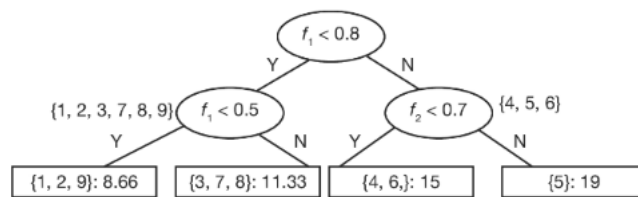
Further,

$$\mu_1 = 10 \text{ and } \mu_2 = 16.33 \quad \mu_1 = 10 \quad \text{and} \quad \mu_2 = 16.33$$

It is possible to compute and show that the best split using f_2 is obtained for the test $f_2 < 0.4$; the corresponding weighted SE is **33.55**, which is larger than the optimal value of **14.9** obtained using f_1 . So, we use the test $f_1 < 0.8$ at the root. Proceeding further in splitting the children nodes, we obtain the DTC shown in Fig. patterns along with the corresponding average target (y) value is indicated. For example, in the leftmost leaf node, the entry is **{1,2,9}: 8.66**, where the subset indicates the collection of patterns 1, 2 and 9; their target values are 8, 8 and 10, respectively, and their average value is

$$\frac{8 + 8 + 10}{3} = 8.66.$$

Decision tree for the regression data set



Now, if we are given a test pattern of the form $(0.3, 0.4)$, then $f_1 = 0.3$, $f_2 = 0.4$. So, we test at the root and go to its left child because $f_1 < 0.8$. Now we take the left child of the left child of the root node; we do this because $f_1 < 0.5$. So, we end up in the leftmost leaf node; this node is represented by its average value of 8.66. the true target ... the value of $(0.3, 0.4)$ is 9. So, the squared error is $(9 - 8.66)^2 = 0.1111$.

Note that for the **KNN regression** in , we predicted a value of **9.33** for the pattern $(0.3, 0.4)$, which again means a squared error of **0.1111**, of course, with a different prediction.

We can abstract the process of building a DT for regression as follows:

- Associate the entire training data set with the root of the decision tree. Select the best feature and best split point based on the weighted SE value of the target (y) values.
- Split the set of examples using the selected test (feature and split point) and associate the resulting subsets with the children nodes of the current node.
- If a child node does not satisfy the termination condition, split it based on best test (feature... and split selected) and repeat steps 2 and 3 till there is no need to split.

The **termination condition** checks for the purity of a node and decides to split if the impurity is above a threshold.

Compute the mean value of the target for each leaf node and use it in prediction by associating this value with the leaf node. Given a test pattern, traverse the DT starting from the root and going down a path till a leaf node is encountered.

The predicted value of the test pattern is the average target value of the leaf node.

Bias–Variance Trade-off

Bias and variance are two important concepts in ML.

Example : Consider the data shown in Table . There are four examples with their respective target values; consider only the first three columns in the table. Let us select the first 3 rows to train the model and the 4th

row for testing.

Let us start with a simple linear model; it is a straight-line fit of the form

$$y=mx+c,$$

where **m** and **c** are the unknowns.

The least-square fit for the first three points gives us the following two equations based on the two unknowns:

$$\sum_{i=1}^3 y_i = 3c + m \sum_{i=1}^3 x_i \Rightarrow 11 = 3c + 3m$$

and

$$\sum_{i=1}^3 x_i y_i = c \sum_{i=1}^3 x_i + m \sum_{i=1}^3 x_i^2 \Rightarrow 17 = 3c + 5m$$

Solving these equations gives us the values:

$$m=3, c=2/3$$

Now if we use these values, the estimated value of **target** for the test pattern with $x=1/2$

$$3 \times 1/2 + 2/3 = 2.16.$$

Note that the fourth column in the table corresponds to the straight-line fit given by y_{sl}

Polynomial fits to illustrate bias and variance

Example	x	Target (y)	y_{sl}	y_{p1}	y_{p2}
1	0	1	$\frac{2}{3}$	1	1
2	1	3	$\frac{11}{3}$	3	3
3	2	7	$\frac{20}{3}$	7	7
4	$\frac{1}{2}$	$\frac{7}{4}$	2.16	1.84	1.87

For this data of three points, we can also fit (overfit) a **degree-3 polynomial**.

In general, we can have **infinite polynomials of degree 3** that can fit the three training patterns. We consider two such polynomials that capture all the three training examples (the first 3 rows) **perfectly**.

The polynomials are:

$$y_{p1} = 1 + \frac{3}{2}x + \frac{1}{4}x^2 + \frac{1}{4}x^3$$

and

$$y_{p2} = 1 + \frac{5}{3}x + \frac{1}{3}x^3$$

The values predicted using these two polynomials are shown in **columns 5 and 6** of the table. Note that y_{p1} and y_{p2} give us the same values as the target y for the training examples. Observe that the 5th and 6th columns are identical and agree with column 3 on the three training examples.

However, on the test point (4th row in the table), they give different values: 1.84 and 1.87, respectively. Now the basic question is which of these models is correct? Note that using three training points, we can fit a degree-2 polynomial uniquely. Starting with a quadratic of the form and plugging in the values of x and y from the first three rows of the table, we get $a = b = c = 1$.

So, the polynomial underlying the data is $1 + x + x^2$; use the 4th row to check that this polynomial exactly satisfies the 4th row also. So, the correct polynomial in this example is of degree-2.

When we use the straight-line fit y_{sl} , we are underfitting the data as we get an error even on the training data, as can be seen from columns 3 and 4 of the table. This is a simpler model.

On the other hand, we can have infinite higher-degree polynomials fitting the first three rows of the table. This is because a degree-3 polynomial fitting the three training examples is **not unique**; we can draw infinite degree-3 polynomials passing through any three points. Of course, there will be a unique degree-3 polynomial **if we use four points**.

We have considered two degree-3 polynomials which overfit the training data and can give different estimates for the target, as seen in columns 3, 5 and 6 of the fourth row of the table. Thus, there is non-zero variance among the target values estimated by the two degree-3 polynomial models, even though they successfully fit the training examples correctly.

So, a linear model underfits the training data. it may make mistakes even on the training data and there is no variance in the estimate of the

target. This is because we have a single linear model fitting the data points using the least-squares and we predict a unique value on the test pattern using such a linear model.

However, degree-3 polynomials overfit the training data and each polynomial gives a different target value estimate, leading to variance in the predicted values.

So, simple models underfit; they have low variance and high bias as they fail to capture even the training data perfectly.

On the contrary, complex models overfit; they commit no errors on the training data and make different predictions on the test data, leading to low bias and high variance.

This phenomenon is known as the bias–variance trade-off.

Making the model simple increases bias, and making the model complex increases variance in predictions. One needs to learn a model that strikes a balance.

Random Forests for Classification and Regression

DT building can be expensive and the resulting tree may not be optimal because of the greedy nature of the building algorithm; these problems prevail significantly in high-dimensional data sets. So, DTCs are not typically used in applications dealing with high-dimensional data. One popular model used to deal with these problems is the random forest (RF). Its features are as follows:

- It employs a collection of randomly generated DTs in prediction. It is called a random forest as it is a collection of trees.
- Each DT makes a prediction and the overall predicted value is based on majority voting in classification and a suitable combination of the values predicted by the DTs.
- The underlying phenomena may be called bagging or bootstrap aggregation. Here,

Bootstrapping means sampling multiple sets of training data. In the case of RF, it works as follows. Given n training patterns in some L -dimensional space, m samples, each of size n data points, are selected with replacement; it is possible for the same data point to occur more than once in a sample.

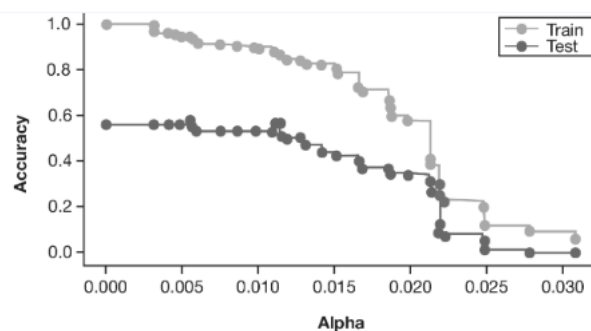
◦ In the RF model, we select a random subset of I ($< L$) features in each sample; so, it is possible that a feature appears in none, one or more

samples. So, each sample may be viewed as an $n \times l$ data matrix. So, for m samples, we have m such $n \times l$ data matrices.

- We construct a DT for each sample. So, each DT is built based on only l features; it is possible that the given set of features is large in size (l is large), but we use only a small number (l) of features for constructing each DT. So, high-dimensional data sets can be handled.

Even though each DT is built greedily, because of multiple DTs, different features may be used in decision making at the root nodes and other decision nodes of the m trees.

Given a test pattern x , we compute the outputs of the m DTs on x and **aggregate** these m outputs to obtain the final decision. In the case of classification, we take a majority of the m class labels and declare it as the class label of the RF model. In the case of regression, the output of the RF model is the average of all the m DT predictions.



Alpha versus accuracy on the Olivetti Face data set (for colour figure, please see *Colour Plate 1*)

Comparison of DT and RF Models on Olivetti Face Data

We compare the performance of the DT and RF models on the Olivetti Face data set. We show the results of using the DTC, by varying α , on the Olivetti Face data set in Fig. 3.

Optimal test accuracy is achieved at a value of $\alpha = 0.011$. At this value, the depth of the DT is 26 and the number of nodes is around 80. At this value of α , the accuracy of the DTC is 0.595 using the GINI index and 0.547 using entropy. The number of training images was 360 and the number of test images was 40.

The RF model was used on the data with 10-fold cross-validation using 360 images for training and the remaining 40 for testing. The

accuracy is shown in Fig. 3.15. Note that the maximum accuracy of **0.967** is obtained when the number of DTs in the RF is 500.

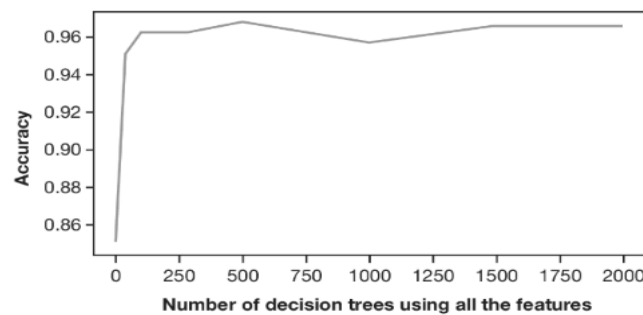


Fig: Number of decision trees versus accuracy using a random forest classifier and all the 4096 features

The Bayes Classifier: Introduction to the Bayes Classifier

The Bayes classifier is an optimal classifier. It operates based on the probability structure associated with the domain of application. It employs Bayes' rule to convert the *a priori* probability of a class into posterior probability with the help of probability distributions associated with the class. These posterior probabilities are used to classify the test patterns; the pattern is assigned to the class that has the largest posterior probability. In case the probability structure is not readily available, the training data is used to learn the probability structure.

Some of the important properties of Bayesian classifiers are as follows:

- It minimizes the probability of error associated with classification.
- It can deal with data that employs both categorical and numerical features.
- It is more of a benchmark classifier having sound theoretical properties. However, in most practical applications, the underlying probability structure is not readily available.
- Some additional constraints on the probability structure are applied to make estimation of the probability structure simpler. For example, the **naïve Bayes classifier (NBC)** is one that assumes that features are independent of each other, given that the data points belong to a class.
- Primarily, there are two schemes for estimating the probabilities associated. One of them depends solely on the data; it is called the maximum likelihood estimate or the frequency-based estimate. The other scheme is more general and it combines application domain knowledge with the data. in estimation; it is called the Bayesian estimation or Bayesian learning of the probability structure.

Probability, Conditional Probability and Bayes' Rule and Inference :

Example : Let us assume that 10,000 people in a community had undergone the COVID-19 test and 50 of them tested positive, while the remaining 9,950 tested negative. In this two-class (binary class) problem, a simple frequency estimate will give us the following values for the probabilities for the two classes:

$$P(\text{positive}) = \frac{50}{10000} = 0.005 \text{ and } P(\text{negative}) = \frac{9950}{10000} = 0.995$$

These probabilities are called **prior probabilities** as they are obtained using domain knowledge

Suppose a new person, from the community, who has not undergone the test, needs to be classified. In the absence of any other information from the person, one would try to use the prior probabilities in decision making; so the new person is assigned a COVID-19-negative label as the probability $P(\text{negative})$ is significantly larger than $P(\text{positive})$ ($0.995 \gg 0.005$). So, invariably every other person in the community who has not undergone the test...

Such a classification is erroneous if the new person is actually COVID-19-positive. This can occur with a probability of 0.005 ($P(\text{positive})$); so, the probability of error is 0.005. Note that every person who is actually COVID-19-negative is correctly .

This is not new; in the KNN classifier, if the value of $k=n_k$, where n is the size of the training data set, and if we have

k_1 (out of k) from the covid-19-positive class and k_2 ($k - k_1$) from the covid-19-negative class, then the probability estimates are:

$$P(\text{positive}) = \frac{k_1}{n} \text{ AND } P(\text{negative}) = \frac{k_2}{n}$$

It is not difficult to see that KNN gives the same result as classification based on prior probabilities.

.2.3 Bayes'Rule and Inference

Let us examine Bayes' rule and inference based on conditional probabilities. We consider some of the terms:

$P(C_i)P(C_{-i})P(C_i)$: prior or initial probability of event C_i, C_{-i}, C_i .

Model of the world (AAA) under each C_i, C_{-i} : $P(A|C_i)P(A \mid C_{-i})P(A|C_i)$

How to infer $P(C_i|A)P(C_{-i} \mid A)P(C_i|A)$, or what is the probability $P(C_i|A)P(C_{-i} \mid A)P(C_i|A)$ — or equivalently how the prior

probability $P(C_i)P(C_i)$ gets updated to the .posterior probability $P(C_i|A)$. Let us consider an example to appreciate these ideas further.

The Bayes Classifier and its Optimality:

The Bayes classifier is a probability-based method used to decide the class of a new data point. It works by choosing the class that has the highest posterior probability after combining prior information with new evidence. If we know the true probability distributions of each class, this classifier gives the lowest possible error. This is why the Bayes classifier is considered optimal.

The classifier is based on Bayes' theorem, which is given by the formula:

$$P(\text{Class} | \text{Data}) = [P(\text{Data} | \text{Class}) \times P(\text{Class})] / P(\text{Data})$$

Example:

Suppose we want to predict whether an email is Spam or Not Spam based on the word 'Discount'. Assume from previous data that:

- $P(\text{Spam}) = 0.6$
- $P(\text{Not Spam}) = 0.4$
- $P(\text{'Discount'} | \text{Spam}) = 0.7$
- $P(\text{'Discount'} | \text{Not Spam}) = 0.2$

Using Bayes' theorem:

$$P(\text{Spam} | \text{'Discount'}) = [0.7 \times 0.6] / P(\text{'Discount'})$$

$$P(\text{Not Spam} | \text{'Discount'}) = [0.2 \times 0.4] / P(\text{'Discount'})$$

We do not need $P(\text{'Discount'})$ because the numerator is enough for comparison. Thus, $0.42 > 0.08$, so the email is classified as Spam. This shows how the Bayes classifier picks the class with the highest posterior probability.

Bayes classifier is OPTIMAL

- It gives the lowest possible error among all classifiers
- No other classifier can beat it in terms of accuracy if we know the true probabilities of the data.
- It uses all available information
- It looks at: The prior probability of each class (how common it is)

- The likelihood of seeing the data for each class
- It combines them intelligently using Bayes rule

Multi-Class Classification:

Multi-class classification is a machine learning method used when a data item can belong to one of three or more classes. Unlike binary classification, where we choose between two options, multi-class classification handles multiple possible outcomes. The idea is to use probability to decide which class a new data point most likely belongs to.

The Bayes approach can also be used for multi-class problems. For each class, we compute the posterior probability using Bayes' theorem and choose the class with the highest value.

Bayes' theorem for class C_i is:

$$P(C_i | x) = [P(x | C_i) \times P(C_i)] / P(x)$$

Example:

Suppose we want to classify a fruit as Apple, Orange, or Banana based on its features like color and size. From past data, we know the following probabilities:

- $P(\text{Apple}) = 0.4$
- $P(\text{Orange}) = 0.35$
- $P(\text{Banana}) = 0.25$

Assume the observed fruit is yellow and long. We have likelihood values:

- $P(\text{features} | \text{Apple}) = 0.1$
- $P(\text{features} | \text{Orange}) = 0.05$
- $P(\text{features} | \text{Banana}) = 0.7$

Using Bayes' theorem (ignoring $P(x)$ for comparison):

- Apple score = $0.1 \times 0.4 = 0.04$
- Orange score = $0.05 \times 0.35 = 0.0175$

- Banana score = $0.7 \times 0.25 = 0.175$

Since Banana has the highest posterior score, the classifier predicts the fruit is a Banana. This demonstrates how multi-class classification works by selecting the class with the highest probability.

Class Conditional Independence and Naive Bayes Classifier (NBC):

The Naive Bayes Classifier (NBC) is a popular and simple machine learning method used for classification. It is based on Bayes' theorem but makes one important assumption: all features are conditionally independent given the class. This assumption is called class conditional independence. Even though this assumption may not hold perfectly in real-world situations, the Naive Bayes classifier still performs surprisingly well in many applications such as spam filtering, sentiment analysis, and medical diagnosis.

Class conditional independence means that once we know the class label, the features do not influence each other. For example, if we are classifying emails as spam or not spam, features like 'contains discount' and 'contains offer' are treated as independent, even if they occur together often in real spam emails. By assuming independence, NBC becomes very simple and fast to compute.

The formula for the Naive Bayes classifier for a class C_i is:

$$P(C_i \mid x_1, x_2, \dots, x_n) \propto P(C_i) \times P(x_1 \mid C_i) \times P(x_2 \mid C_i) \times \dots \times P(x_n \mid C_i)$$

Example:

Suppose we want to classify a message as Spam or Not Spam using two features: the presence of the words 'Free' (x_1) and 'Offer' (x_2).

Assume the following probabilities from past data:

- $P(\text{Spam}) = 0.6$
- $P(\text{Not Spam}) = 0.4$
- $P(\text{'Free'} \mid \text{Spam}) = 0.7$
- $P(\text{'Offer'} \mid \text{Spam}) = 0.8$
- $P(\text{'Free'} \mid \text{Not Spam}) = 0.2$

- $P(\text{'Offer'} \mid \text{Not Spam}) = 0.1$

Using Naive Bayes:

Score for Spam = $0.6 \times 0.7 \times 0.8 = 0.336$

Score for Not Spam = $0.4 \times 0.2 \times 0.1 = 0.008$

Since the Spam score is higher, the NBC classifies the message as Spam.

This example shows how Naive Bayes multiplies the likelihood of each independent feature along with the class prior to make the final prediction.